

GROUP TECH BRIEF / TIBO TALK SERIES #04

Prompt Engineering 会退潮吗

Tibo x Silicon Valley Girl: AI Agents for Everyone

视频播客 / 演示型访谈 • 29 分钟 • 会谈日期 2026.05.25

一句话结论

这场最鲜明的判断是：Agent 大众化之后，竞争力会从“会写复杂 Prompt”迁移到 Context、Memory、权限、工作流封装和正确提问；用户不再学习软件，软件开始适应用户。

四个最值得关注的信号

01 Packaging > 配置 Cron、工具连接、后台任务正在从极客配置变成自然语言产品能力。	02 Cloud memory 跨设备 Agent 需要把文件、记忆和长期 context 从单机迁到云端。
03 Auto-review 第二 Agent 监督第一 Agent，让更长时间、更高权限的自治成为可能。	04 Prompt 退居二线 未来收益不应与 prompt 技巧强绑定，而应来自系统自动理解用户和长期目标。

说明：下文将“访谈观点”“技术解读”“方向推演”分开呈现；预测不视为正式产品承诺。

OVERVIEW

访谈背景与章节地图

受访者：Thibault “Tibo” Sottiaux，OpenAI Core Product / Codex 相关负责人。

对谈方：Marina Mogilko（Silicon Valley Girl）

形式：视频播客 / 演示型访谈；时长：29 分钟

- 访谈观点
- 技术解读
- 方向推演
- 外部核验

章节地图 / 讨论顺序

时间 / 顺序	主题
01:46	为什么现在 Agent 才开始大众化
02:41	“每 12 小时跑一次研究并发 PDF” 这类自动化
03:58	Auto-review：第二 Agent 审查第一 Agent 动作
04:58	本地文件 -> 云端 Memory / Files
09:10	人仍对代码与系统负责，不能外包理解
11:38	Vibe coding 与长期可维护性
22:48	大量非技术任务
26:44	3-5 年：AI 变成 ambient intelligence
27:26	“问对问题” 比 prompt engineering 更重要

阅读方法

先看 Tibo 明确表达的系统设计原则，再看这些原则对 Agent Runtime、Serving、Sandbox、Memory、Multi-agent 或软件工程流程的含义。不要把方向判断直接当作已经落地的功能。

CORE IDEAS

1. 为什么 Prompt Engineering 不是长期护城河

- 今天很多 AI power user 的优势来自会写 prompt、会组织文件、会配置 tool、会搭 cron 和 agent workflow。Tibo 的判断是，这些复杂度最终应该被产品吸收。
- 他用“裁缝”类比未来 Agent：系统先理解你，再主动给出适合你的帮助。用户不应该为了机器而学习一套固定操作语言。
- 因此真正长期有价值的用户输入更接近：目标是什么、什么算好、你的偏好是什么、什么不能做、什么时候需要你介入。

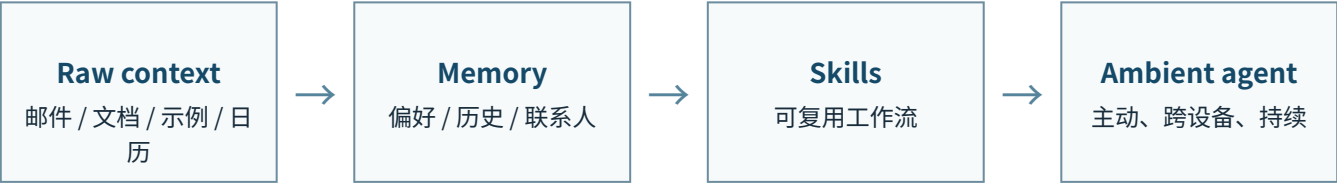
核心判断：这场最鲜明的判断是：Agent 大众化之后，竞争力会从“会写复杂 Prompt”迁移到 Context、Memory、权限、工作流封装和正确提问；用户不再学习软件，软件开始适应用户。

2. Context / Memory 才是个人 Agent 的“复利资产”

- Tibo 当时仍把很多 notes 放在本地文件夹里，再让 Agent 整理；但他明确判断这些内容会迁到云端，形成跨设备的同一个 Agent。
- 这意味着个性化的核心不再是一个 system prompt，而是持续积累的状态：历史决策、项目资料、写作样例、联系人关系、常用流程和授权。
- 工作流一旦稳定，还可以封成 Skill / automation，从一次性 prompt 变成可重复执行资产。

SYSTEM VIEW

把访谈观点翻译成系统结构



这里最容易误解的一点

Agent 的“能力”并不等于裸模型能力。对真实任务而言，更接近：

$$\text{Agent Utility} = \text{Model} \times \text{Context} \times \text{Tools} \times \text{Runtime} \times \text{Safety} \times \text{Verification}$$

任何一项接近 0，端到端体验都会坍塌；这也是为什么模型 benchmark 提升并不总能等比例转化为产品收益。

工程上应该关注的闭环

Observe	任务 trace、tool call、权限、失败位置、人工介入、最终结果。
Diagnose	区分是模型、Harness、环境、数据、延迟还是安全策略导致失败。
Improve	把问题放回最合适的层修，而不是默认继续堆 prompt 或规则。
Verify	用可重复 eval 证明端到端成功率、成本和 latency 真正改善。

AI INFRA TAKEAWAYS

对 Agent Platform 的启发

Memory quality	需要 freshness、冲突处理、来源追踪、遗忘/更新机制；“记得越多”并不自动等于更好。
Skill compiler	自然语言工作流 -> 可测试、可版本化、可观测的执行图，是把 power-user 能力大众化的关键。
Cross-device state	同一个 Agent 需要云端 identity / state / permission，而不只是同步聊天记录。
Human responsibility	Agent 产出越多，verification debt 越容易积累；“没读过但能跑”的代码在规模化场景依旧危险。
Tool latency	当 prompt 成本下降，真正体验差距会更多来自工具连接速度、数据权限和执行成功率。

给组内讨论的一个统一框架

把每项 Agent 优化问成四个问题：①端到端成功率提升多少；②wall-clock 缩短多少；③总算力/Token/工具成本增加多少；④风险与人工验收成本是否变高。单看模型分数、Kernel 峰值或单 Token 价格都不够。

$$\begin{aligned} T_{total} = & T_{queue} + T_{prefill} + T_{decode} + T_{tool} + T_{network} \\ & + T_{environment} + T_{verification} + T_{human_wait} \end{aligned}$$

WHAT TO BELIEVE / WHAT TO DO

需要打折看的部分

- “几个月内” “6-9 个月显著改善” 等属于个人预测，不是交付承诺。
- 第三方转写存在少量产品名误转写；本报告依据上下文纠正明显拼写，但不据此推断未明确表达的功能。
- “Prompt engineering 消失” 更准确地说是“显式提示技巧被产品封装”，而不是目标表达和约束设计不再重要。

给组员的三个可执行动作

1. 把团队常用的一次性 prompt 找 3 个，改造成带输入/输出 schema、验证规则和版本号的 Skill。
2. 给 Memory 增加 source + timestamp + confidence，避免错误记忆长期污染。
3. 评估 Agent 产品时，单独统计“人工配置时间”和“任务执行时间”，看包装层是否真的降低总成本。

本场会谈的最终一句话

这场最鲜明的判断是：Agent 大众化之后，竞争力会从“会写复杂 Prompt”迁移到 Context、Memory、权限、工作流封装和正确提问；用户不再学习软件，软件开始适应用户。

原始资料

<https://podcasts.apple.com/us/podcast/head-of-chatgpt-codex-ai-agents-for-everyone-are/id1819090545?i=1000769465181>

<https://podscripts.co/podcasts/silicon-valley-girl-ai-tech-and-career-growth/head-of-chatgpt-codex-ai-agents-for-everyone-are-here-thibault-sottiaux-openai>

整理日期：2026-09-10。报告为技术阅读笔记，不代表 OpenAI 官方路线承诺；产品状态以最新官方文档为准。